

Year 2000 — a Systems Integration challenge

R Lewis

BT, along with virtually every other IT-dependent business world-wide, is tackling a problem which is quite unique and if not corrected could be disastrous. The problem, sometimes known as the 'Year 2000 bug' or 'millennium time bomb', has been caused by the use of two digits to represent the year in the majority of our systems and applications. The problem is technically not difficult to fix but the volume of changes occurring, and the need to potentially test every system to ensure that it is 'year 2000 proof', presents unique and challenging difficulties for integration and testing. Why this is the case, what problems need to be addressed, and an overview of some of the proposed integration and test strategies to tackle these problems, is the subject of this paper.

1. Introduction

If you believe all the hype that has appeared in numerous newspaper and magazine articles, then at midnight 31 December 1999 you will be cowering under your kitchen table clutching a carrier bag containing your life savings. Your home will be in darkness, the car won't start, and passing aeroplanes will fall from the sky. The world economy will be in ruins as a wave of disaster sweeps westwards across the globe as billions of computers wake up and think it is 1900 again.

This scenario may seem rather far-fetched but, if nothing is done to fix the 'millennium time bomb', as it is often referred to, then it might not be so far from reality. Thankfully, BT and most other IT-dependent companies have realised that they need to take this very seriously and have set up projects within their organisations to tackle the problem.

The 'Year 2000' problem originated in the 1960s and 1970s when memory and disk space were expensive and programmers had to write software to fit within very tight hardware and software constraints. Using two digits to indicate the year instead of four (assuming a prefix of 19 in front) became normal practice saving considerably on memory and storage costs. This practice continued at the expense of 'future proofing' the systems. It was thought at the time that systems would probably be replaced during the 1990s when memory and disk costs would no longer be an

issue. Clearly this has not happened and the industry has been left with a huge problem on its hands.

Although predominantly a computer-application problem, there are also large numbers of embedded software systems which are also at risk. We do not, however, have to wait until the year 2000 to encounter the problem — it is already with us. For example, retailers are already experiencing problems authorising credit cards with expiry dates on or after 2000. Marks & Spencer's computer system recently ordered new cans of corned beef to be discarded because their sell-by date was post 2000. A 104-year-old lady in Reading has received a nursery school voucher because the computer system believed her to be 4 years old!

2. So what is the problem?

Along with most other IT companies, many of BT's systems use two digits for the year (i.e. dd/mm/yy) and as the 1 January 2000 approaches, software that sorts information or performs calculations using two digits for the year is at risk of failure. Unless the programs have been coded to recognise 00 as occurring after 1999, the program will fail, or even worse, carry on but give incorrect results. The problem is further aggravated by the fact that the year 2000 is a leap year and it is likely that not all systems will recognise it as such.

The problem is not difficult to fix and presents little in the way of technical or intellectual challenge. Put simply, each system, its interfaces (internal and external), data, and business processes have to be assessed for exposure to the century date change. If there is a problem then a fix has to be implemented, tested, integrated, end-to-end tested and then deployed. This may sound straightforward enough, but multiply this by the number of systems in BT's inventory and you can begin to appreciate the volume of changes that will need to take place. The fixed deadline and the reliance on testing makes the task particularly challenging and unique. Slipping the deadline is simply not an option and it is unfortunate that the industry's track record for delivering IT projects on time is far from good — a recent survey reported that 80% of all computer projects are delivered late.

3. How can the problem be fixed?

The BSI has produced a compliance statement [1] that states that: 'Year 2000 conformity shall mean that neither performance nor functionality is affected by dates prior to, during and after the year 2000'. In particular, the following rules will apply:

- no value for current date will cause any interruption in operation,
- date-based functionality must behave consistently for dates prior to, during and after Year 2000,
- in all interfaces and data storage, the century in any date must be specified either explicitly or by unambiguous algorithms or inference rules,
- year 2000 must be recognised as a leap year.

The two most common solutions for achieving compliance are to either expand the year representation to four digits, and thereby remove any possible ambiguity in interpretation, or add programming logic to determine whether the date is before or after the year 2000.

3.1 Expand the year to four digits

This is the most enduring solution and is favoured where a more robust or permanent 'fix' is required. Little or no changes to program logic are required but the down side is that it requires changes to data, files and interfaces. There is also a resultant increase in the amount of integration and system testing required. Systems will also need to be integrated and deployed at the same time or bridges/ filters used to handle the differences at the interfaces during transition.

3.2 Windowing technique (fixed or sliding)

This solution requires additional program logic to determine whether the date is before or after 2000. For example:

if $yy \geq 50$ then year = 19yy

if $yy < 50$ then year = 20yy

This solution will only work within a window of 100 years and therefore has a limited life. Depending on the pivot date, 50 in the above example, the solution will cease to work when the upper threshold is reached. There is a sliding window variant of this solution that does allow the window to be advanced. The technique has obvious limitations but its major attraction is that data and interfaces are considerably less impacted. Less integration testing will therefore be required and the need for bridges significantly reduced. However, because it necessitates coding changes then it does increase the risk of introducing new defects and therefore the need for increased regression testing.

4. Impact on test and integration

Integration and testing is expected to be the biggest component of the overall cost of fixing the Year 2000 problem. Many authorities on the subject and companies who have already completed Year 2000 projects report that as much as 60% of the total expenditure was spent on testing.

This is not surprising given that testing will be the only real means of proving that a system or platform is compliant. Even systems already considered to be compliant will have to undergo a test to verify compliance at some stage. Many of the computing platforms and system software upon which BT's applications currently run are not yet compliant and so additional testing will be needed when compliant releases are received from vendors. It is reasonable to predict, therefore, that integration and testing will be on the critical path for most projects.

Several difficulties will arise during the testing stages for Year 2000. The main areas of concern are the ability to create suitable test environments with time-shifting to allow applications to run through the Year 2000 test scenarios, creating and ageing the test data, the increased need for test automation, and planning and sequencing development and integration activities.

Complex inter-dependencies exist between many of the systems and therefore design and implementation decisions for an individual system cannot be made in isolation. The impact on other systems, and platforms, to which the system

has interfaces, has also to be considered when choosing solutions. With this in mind, design authorities/development managers and systems integration/VV&T managers have a key dual role in determining appropriate design solutions and assessing impact on integration and test. Careful co-ordination and sequencing of developments, integration and deployment will be essential if the most efficient use of time and resources is to be achieved.

Because of the need to roll forward the system clock and to age data for testing purposes, it may not be practicable or desirable to use existing test environment facilities. There may be many side effects to advancing system dates. Licenses may suddenly expire, logons become disabled, date triggered housekeeping routines activated, etc. Such interference to business-as-usual activities, and possible contamination of existing data, will be unacceptable. For this reason, and to provide additional test environment capacity to cater for increased demand, a dedicated Year 2000 test environment is being set up.

5. So much to do — where do you start?

On current estimates, BT has over 1200 known systems in its inventory, any or all of which may need fixing. Given that time is against us and there are finite resources available to carry out the work, it may not be feasible to fix and test all systems in the time remaining. Indeed it may be decided that some systems will be decommissioned or replaced rather than be made compliant. It is essential, therefore, that business criticality is assessed and prioritised for each system. Deciding what systems have to be fixed first, those that do not have to be fixed, and those to be decommissioned or replaced, is the essence of an approach called 'triage'.

5.1 The triage approach

The term 'triage' was first used in World War II to prioritise the limited medical resources for treating the injured based on the nature and severity of their injuries, i.e. those that would only survive if given medical attention were treated first. In the context of the Year 2000 problem, 'triage' is now being used to prioritise fixing the problem on those systems that are critical to the operation of the business.

In BT's case, this means those systems that support the product line revenue streams, both in revenue generation and collection, that provide the business with legal and regulatory information, and those that support future strategic initiatives (the horizon industry). The triage principle will drive both the placement of development effort and the upgrade of supporting technical infrastructure.

So what does this mean in practice? Every component in BT's portfolio, be it an operational support system (OSS), an application, a supporting hardware or software platform, a switch, a network application, or CPE (customer premises equipment), will need reviewing to determine the extent of the Year 2000 problem and cost of repair.

Systems are then classified in terms of their criticality and priority ranging from critical (level 1), major (level 2), minor (level 3) to low (replace or decommission).

5.2 Identifying (integration) critical systems

An 'integration-led' approach has been used to identify those systems that are business or operationally critical (level 1). For example, a number of systems are used across several of the key service platform architectures. Examples of these systems are CSS, COSMOSS, NMP, SDP, CAMSS, etc [2]. Such systems are therefore critical because a large number of BT's key products, services, or business processes will be impacted if these systems fail. In addition, because these systems are pivotal to several of the key integration programmes, it is vital that they are delivered early into integration to allow the rest of the components to be integrated and end-to-end tested. A matrix of systems mapped to service platforms, as illustrated in Fig 1, has been used to determine the list of critical systems.

Using this approach, 70 systems have so far been identified as level 1 and 238 as level 2 systems, i.e. providing input to a level 1 system(s). Work is continuing on the classification of the remaining systems in the inventory and assigning criticality levels. Systems with the highest criticality levels will therefore provide the initial focus for development and integration planning.

6. Testing strategy

Much of the work to convert, integrate and test systems will be based, as far as possible, on 'business as usual'. To assist teams with formulating verification, validation and testing (VV&T) strategies, a VV&T guidance document [3] has already been produced which is based on standard practice as defined in VV&T Handbook [4] and the Systems Integration Handbook [5].

Essentially, any changes required to implement Year 2000 compliance should not introduce new functionality. Therefore, testing will be a combination of regression testing to demonstrate that the system functions as it did prior to conversion (current dates), and executing tests and creating test data (future dates) to demonstrate that it meets the compliance criteria.

		service platform supported									
system name	operational platform	PSTN/ISDN	DSN	F-Net	IN	billing	card	private ccts	international	SMDS/FBS	
CSS	MVS.IDMS	Y	Y	Y	Y	Y	Y	Y			
COSMOSS	MVS/DB2		Y	Y	Y			Y	Y	Y	
NMP	ICL/Infomix	Y		Y		Y	Y		Y		
SDP	HP9000/Oracle	Y		Y		Y	Y				
CAMSS	MVS/IDMS		Y	Y	Y		Y		Y	Y	
WM	Unix/Oracle	Y			Y		Y	Y			
OMC	VAX/Oracle	Y		Y				Y			
ServiceView	PC/workstation		Y	Y	Y					Y	
NOMSI	Unix	Y						Y			

Fig 1 Identifying critical systems in the inventory.

6.1 Typical compliance tests

Testing scenarios for Year 2000 readiness will depend on the specific system environment and the application under test. Some of these scenarios could also be used to test for exposures to the Year 2000 problem as part of an initial impact analysis.

In formulating test scenarios consideration needs to be given to the processing time horizon for that system. For example, a system may need to process information forward into the next century (progressive) or look back into this century from a post-2000 date (regressive):

- current dates — after the system or application has been modified, testing should demonstrate that the system performs as it did before (regression test),
- processing over different time horizons — testing forwards and backwards processing across the century date change (dates prior to and including 31 December 1999 and dates after and including 1 January 2000),
- financial year — testing forwards and backwards across the financial year (dates prior to and including 31 March 2000 and dates after and including 1 April 2000),
- leap year — testing specifically across the dates 27, 28 and 29 February and 1 March 2000 (29 February is a valid date),
- day of the week — testing that 1 January 2000 has the correct day of the week, Saturday (it is possible that a Monday is assigned incorrectly which is the day for 1 January 1900),

- special meaning dates — testing that dates have not been used as flags or to indicate special meanings, e.g. 31/12/99 (never expire flag), 9/9/99.

6.2 Creating and ageing test data

Running tests to demonstrate compliance will require changes to the input and output data, particularly if four-digit date expansion is used. Running tests at a forward date will both require modified input data and generate future-dated output data.

The ability to easily create and modify test data and the use of regression test/comparison tools will be a key enabler in achieving the test coverage and the productivity required.

Fig 2 illustrates a proposed test strategy that utilises existing or newly created baseline input data and results. This baseline is then used to regression-test the converted system and subsequently modified to create test data for the different Year 2000 testing scenarios.

The four main test run steps (as shown in Fig 2) are as follows.

- Step 1 Capture initial baseline — current test data is used with unmodified application to create a baseline set of output results.
- Step 2 Application modified, current dates — previous test data is used (with modifications to date format if required, e.g. expand century); results generated should be the same as the baseline except for any changes to century format.

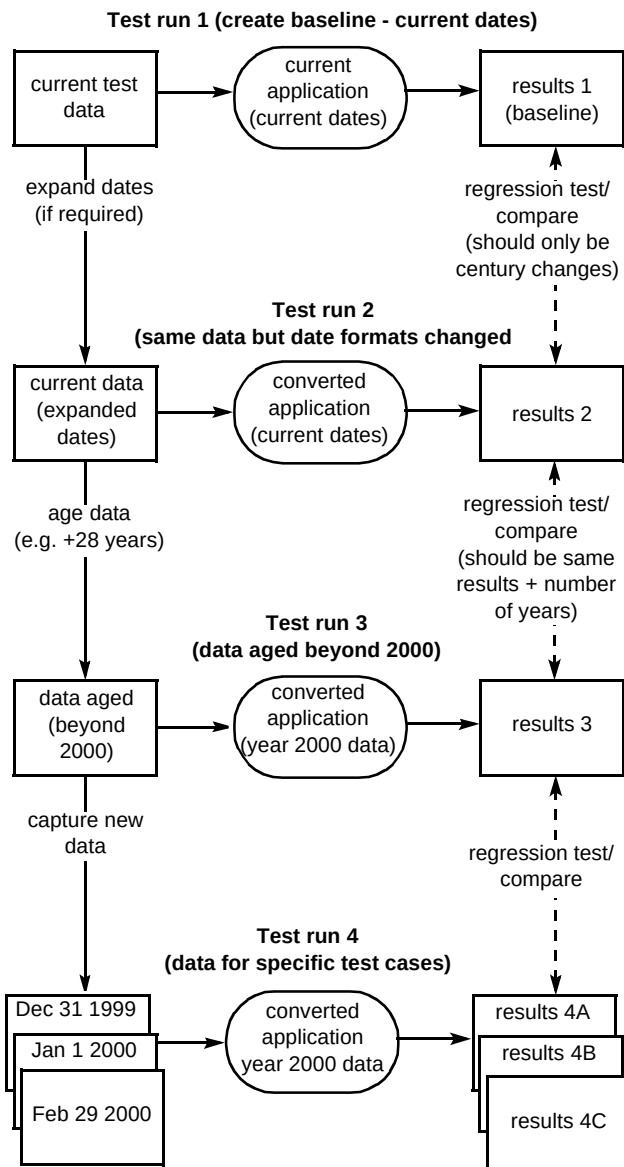


Fig 2 Date ageing and regression test strategy.

- Step 3 Modified application, Year 2000 dates — input data is aged 28 years (all dates including leap year repeat every 28 years); results should be the same as previous baseline + 28 years.
- Step 4 Run specific test scenarios — new data or modified input data is used to run tests at required dates (this can be achieved by using production data with dates at 1971/72 and applying the 28 rule to create 1999/2000 scenarios and so on).

6.3 Use of tools

This approach relies heavily on test automation. Indeed it is unlikely that the volume of testing required for Year 2000 is achievable within the time-scales without extensive use of tools. Clock simulators, test coverage analysers, test

data generators, date ageing tools, capture/replay tools, code analysers, data analysers (to check for instances of date), data comparators, etc, should all be in a tester's tool bag. These tools are readily available and it is imperative that they are used.

6.4 Simulating forward dates

Changing the dates to facilitate testing can be achieved by either altering the system clock or by running special tools to simulate dates for a particular application. Changing system clocks will affect other applications also running on that platform which has obvious disadvantages. A date simulation tool can be used to isolate the effects of changing the machine date to that application. A combination of the two techniques will be used in practice. For example, for unit or sub-system testing, a date simulation tool may be sufficient but for integration and end-to-end system testing, where it is necessary to test operational running, changing the system clocks will be necessary.

However, it must be noted that when testing in these situations some system resources and functions are time-sensitive and may be activated or de-activated when the system clock is changed. For example, automatic house keeping routines may delete apparently old files or software licences may suddenly expire. Without careful planning this could cause the loss of system resources and/or functions, some of which might prove very difficult and time-consuming to recover.

6.5 Performance/stress testing

When testing Year 2000 changes, it may be necessary to test that the existing resources can handle the normal and abnormal volumes of transactions after the restructuring of the code, and possible expansion of the data fields. For example, additional CPU cycles will be needed if additional program logic is introduced for the two-digit 'windowing' solution, and additional disk space will be needed, with increased disk access time, when four-digit date expansion is used.

The use of bridges in particular, may introduce additional processing overheads which could adversely affect performance while they are being used.

7. Pulling it all together

7.1 Integration and test planning

V&T strategies and integration plans must be produced early in the Year 2000 project life cycle. The dates

when systems are required for integration will be used as a driver for the development plans. Where the delivery dates for systems are already fixed, then it will be necessary to order the integration sequencing accordingly. The date when integration and end-to-end test is completed must allow sufficient time for roll-out by the deadline. All this will be largely business as usual but the volume of testing required, the potential for overloading the testing phase if systems are delivered late, and the fixed deadline, will require plans which are integrated, resourced and co-ordinated.

During the latter part of 1997 and 1998, test environments will be in heavy demand and scheduling access and availability will be a vital and centrally managed activity.

7.2 Achieving compliance

Each application or component, whether it is already compliant or requires modification to achieve compliance, will require certification that it is compliant. In most cases evidence of compliance will be achieved through system testing. Where a system or component is handed over for integration with other systems or components, then evidence of successful completion of this test will form part of the quality gate, hand-over criteria, or acceptance test.

Systems will typically have a number of dependent parts such as the computing platform, third-party software, protocols, etc. All of which must be proven to be compliant before the system itself can be regarded as such.

The final stage of compliance will be achieved after the system has been successfully integrated and end-to-end tested. Where there is a user or customer acceptance test, then this may constitute a further stage of compliance sign off.

Therefore, compliance will typically be achieved in three stages of what can be regarded as a 'compliance life cycle':

- infrastructure compliance — the computing platform and system software on which the application runs is demonstrated to be compliant,
- application compliance — the system or application is tested for compliance (running on compliant hardware/system software),

- integration/end-to-end compliance — the system or application is integrated with other systems/applications and end-to-end system tested to verify overall compliance.

Achievement of compliance at each stage can then be tracked via plans and reporting mechanisms for each system in the inventory. The level of monitoring and tracking and the sign-off authority for compliance will depend on the level of criticality assigned.

The Configuration Management Environment (CME) [6] will be used to support this process.

8. Conclusions

The Year 2000 problem constitutes the single most expensive computer 'bug' in human history. It is ironic that a problem which will cost the IT industry hundreds of billions of pounds to correct was created by the industry in the first place.

Integration and testing will be the biggest component of the overall cost of fixing the Year 2000 problem since the only real way of knowing that a system is compliant is to test it. A high volume of changes to BT's systems is predicted; this will place heavy demands on test and integration resources.

Because of the unique and pervasive nature of the Year 2000 problem and the fixed deadline, conventional approaches to VV&T will need to be adapted — testing for the Year 2000 may well prove to be the 'ultimate regression test'.

Further general information on the Year 2000 can be found on World Wide Web [7].

References

- 1 British Standards Institute: 'A Definition of Year 2000 Conformity Requirements', DISC PD2000-1
- 2 Stockman S and West S M (Eds): 'Operational support systems', BT Technol J, 15, No 1, (Special issue) (January 1997).
- 3 Lewis R: 'Millennium VV&T Guidance', Issue 1 (October 96).
- 4 BT: 'VV&T Handbook', Issue 3 (1996).

- 5 BT: 'System Integration Handbook', Issue 3 (1996).
- 6 Moor S R, Gunne-Braden J and Gleen K E: 'Enterprise configuration management — controlling integration complexity', BT Technol J, 15, No 3, pp 61—72 (July 1997).
- 7 Year 2000 Information Centre: <http://www.year2000.com/cgi-bin/clock.cgi>



Ray Lewis moved to BT Laboratories in 1979 after spending his early career in Liverpool and Manchester. After working on the development of computer aided software tools, he specialised in VV&T technology and standards and was responsible for the production of the VV&T Handbook.

He was manager of the Billing and Charging Integration unit and recently became the Systems Integration Manager for the Year 2000 programme.

